

3D Medical CLIP

Implementation Appendix v1.1

依赖架构: Architecture Specification Bundle v2.4.5

文档性质: Mandatory Implementation Specification

状态: implementation_ready

语言: 中文

Research and demonstration use only. Not for clinical diagnosis or treatment decisions.

1. 文档定位

本附录将 Architecture Specification Bundle v2.4.5 转换为可执行的实现约束。它不改变架构边界，只补齐实现时必须固定的事务、数据模型、状态规则和验证要求。

项目	内容
当前版本	1.1
替代版本	Implementation Appendix v1.0
是否改变 Architecture Bundle	否
实现状态	implementation_ready

1.1 修订范围

- Dead-letter Command 持久协议
- 重复消息与当前 Lease Revision 严格绑定
- Recovery Window 与独立重试预算
- GPU Startup Nonce 和两阶段启动握手
- Workspace Source Reference 与 History Snapshot Reference 分离
- Chunk Verification Seal Token
- lightweight / full_archive / re_executable 三类 History Profile
- 跨文件系统 Storage Reservation
- Backup Maintenance 完整状态机与 Fencing
- Qdrant Collection Content Digest
- Clause-level Implementation Conformance

2. 条款编号与符合性

所有强制实现要求必须具有稳定 Clause ID，格式如下：

```
IMP-{DOMAIN}-{NUMBER}
```

示例：IMP-EXEC-001, IMP-HIST-004, IMP-BACK-003

Implementation Conformance Report 必须逐条报告 Clause 状态，不得只按整份规范填写“已实现”。

3. Dead-letter Command Protocol

IMP-EXEC-001：不得静默丢弃异常消息

- Command 不存在
- Model Job 不存在
- Command 与 Model Job 绑定不一致
- Command Schema 不受支持
- Payload 无法解析
- 消息指向错误 Generation
- 恢复后出现幽灵消息

```
detect invalid message
-> persist dead-letter record
-> write audit event
-> ACK original queue message
```

3.1 dead_letter_commands

```
CREATE TABLE dead_letter_commands (
  id UUID PRIMARY KEY,
  original_message_id VARCHAR(256),
  queue_name VARCHAR(128) NOT NULL,
  command_id UUID,
  model_job_id VARCHAR(128),
  raw_payload JSONB,
  command_schema_version VARCHAR(64),
  delivery_attempt INTEGER,
  error_code VARCHAR(128) NOT NULL,
  error_message TEXT,
  detected_by VARCHAR(128) NOT NULL,
  detected_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  resolution_status VARCHAR(32) NOT NULL DEFAULT 'unresolved',
  resolved_by VARCHAR(128),
  resolved_at TIMESTAMPTZ,
  resolution_note TEXT
);
```

状态	说明
unresolved	尚未处理
acknowledged	已确认但未重放
replayed	经管理员批准后重放
discarded	明确废弃
resolved	已完成处置

IMP-EXEC-002：Dead Letter 不得自动重放

- 必须由管理员显式触发
- 必须确认原 Command 与 Model Job 状态
- 必须验证 Schema 兼容性
- 必须写 Audit Event
- 必须保存 Resolution 记录

4. 重复 Queue Message 严格验证

IMP-EXEC-003：安全重复消息条件

```
command.aggregate_id = model_job.model_job_id
AND command.command_generation = model_job.current_command_generation
AND command.acquired_lease_revision = model_job.execution_lease_revision
AND command.lease_owner_instance_id = model_job.worker_instance_id
AND model_job.execution_lease_expires_at > NOW()
AND model_job.supervisor_heartbeat_at > NOW() - heartbeat_stale_threshold
```

任一条件不成立时不得执行，也不得根据最终状态“猜测”正确性。应将 Job 标记为 recovery_required，并交由 Reconciler。

IMP-EXEC-004：Generation 处理

情况	行为
消息 Generation 小于当前值	旧 Command -> superseded, ACK, 不执行, 写 Audit
消息 Generation 等于当前值	继续 Lease/重复消息判断

情况	行为
消息 Generation 大于当前值	视为控制面异常，进入 Dead Letter

5. Recovery Window 与重试预算

IMP-EXEC-005：恢复计数必须包含时间上下文

```
ALTER TABLE command_outbox
ADD COLUMN recovery_window_started_at TIMESTAMPTZ,
ADD COLUMN last_recovery_at TIMESTAMPTZ,
ADD COLUMN consecutive_recovery_failures INTEGER NOT NULL DEFAULT 0,
ADD COLUMN total_recovery_attempts INTEGER NOT NULL DEFAULT 0;

recovery_policy:
  window_seconds: 1800
  max_consecutive_recoveries: 3
  max_total_recoveries: 10
  stable_execution_reset_seconds: 300
```

IMP-EXEC-006：稳定执行只重置连续失败

稳定运行超过阈值后，可以将 consecutive_recovery_failures 清零，但不得清除 total_recovery_attempts、审计历史或 Attempt 历史。

IMP-EXEC-007：不同重试预算不得混用

计数	含义
dispatch_attempts	Dispatcher 发送尝试
delivery_attempts	Broker 投递次数
lease_recovery_attempts	Lease 失效后的恢复次数
execution_attempts	GPU Worker 实际启动后的执行失败次数
application_attempt_number	业务级新 Attempt 次数

6. GPU Startup Nonce 与 IPC 验证

IMP-EXEC-008：每次启动必须生成唯一挑战值

```
worker_startup_nonce = cryptographically random 128-bit value
child_process_uuid = UUIDv4

{
  "model_job_id": "mjob_001",
  "execution_lease_revision": 7,
  "worker_startup_nonce": "...",
  "child_process_uuid": "...",
  "message_sequence": 1
}
```

IMP-EXEC-009：Supervisor 必须验证

- Model Job ID
- Lease Revision
- Startup Nonce
- Child UUID

- Message Sequence 单调递增
不匹配的消息必须丢弃并记录；超过阈值时终止子进程。

7. GPU 两阶段启动握手

IMP-EXEC-010：startup_ready 不等于执行开始

```
spawn child
-> child initializes IPC/runtime
-> startup_ready
-> supervisor persists execution_started
-> begin_execution
-> child starts model loading
```

子进程状态	含义
created	进程对象已创建
initializing	初始化 IPC 和运行时
startup_ready	已准备等待授权
waiting_for_begin	等待持久事务完成
executing	开始模型加载和执行
stopping	停止中
exited	已退出

IMP-EXEC-011：持久事务成功后才能开始

1. 锁定 Command 和 Model Job
2. 验证当前 Lease
3. Command -> execution_started
4. Attempt -> running
5. Job -> loading_model
6. 保存 PID、Child UUID、Nonce Hash
7. 写 Event Outbox
8. 提交事务
9. 提交成功后发送 begin_execution

IMP-EXEC-012：持久更新失败

- 不得发送 begin_execution
- 子进程保持等待
- 等待超时后退出或被终止
- Supervisor 进入恢复或失败流程

8. 错误分类

IMP-EXEC-013：错误码必须预先分类

```
error_policy:
  retryable:
    - REDIS_TEMPORARILY_UNAVAILABLE
    - POSTGRES_CONNECTION_LOST
    - QDRANT_TEMPORARILY_UNAVAILABLE
    - MODEL_SERVICE_RESTART
    - GPU_RESOURCE_TEMPORARILY_UNAVAILABLE
    - QUEUE_DELIVERY_TIMEOUT
  final:
    - EXECUTION_CONTRACT_INCOMPATIBLE
    - INVALID_NIFTI
    - DATASET_WITHDRAWN
    - DATASET_INTEGRITY_FAILED
    - CHECKPOINT_HASH_MISMATCH
    - UNSUPPORTED_RESULT_SCHEMA
```

未分类错误默认进入 manual_review_required，不得自动判定为 failed_final。

9. Source Reference 与 Snapshot Reference

IMP-HIST-001：使用两个独立生命周期引用

引用	保护对象	释放时机
Workspace Source Reference	原 Workspace 文件	Snapshot Manifest 完成并建立 Snapshot Reference 后
History Snapshot Reference	History 保存所用 Snapshot	History Ready 或安全清理完成后

9.1 history_snapshot_references

```
CREATE TABLE history_snapshot_references (
  id UUID PRIMARY KEY,
  snapshot_reference_id VARCHAR(128) UNIQUE NOT NULL,
  save_operation_id VARCHAR(128) NOT NULL,
  snapshot_manifest_sha256 VARCHAR(64) NOT NULL,
  snapshot_path TEXT NOT NULL,
  reference_status VARCHAR(32) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  released_at TIMESTAMPTZ,
  FOREIGN KEY (save_operation_id)
    REFERENCES history_save_operations(save_operation_id)
    ON DELETE RESTRICT
);
```

IMP-HIST-002：引用转换顺序

```
create workspace source reference
-> create snapshot
-> finalize snapshot manifest
-> create history snapshot reference
-> release workspace source reference
-> write/verify history
-> release snapshot reference
-> cleanup snapshot
```

10. Chunk Verification Seal Token

IMP-HIST-003：验证不得依赖超长事务

```
ALTER TABLE history_artifacts
ADD COLUMN verification_generation BIGINT NOT NULL DEFAULT 0,
ADD COLUMN verification_started_at TIMESTAMPTZ;

UPDATE history_artifacts
SET storage_status = 'verifying',
    verification_generation = verification_generation + 1,
    verification_started_at = NOW()
WHERE id = :artifact_id
AND storage_status = 'writing'
RETURNING verification_generation;
```

IMP-HIST-004：Seal 后禁止新 Chunk

只有 storage_status=writing 时允许插入 Chunk。进入 verifying 后，验证器可使用多个短事务分页读取并流式计算 Hash。最终短事务必须再次验证状态、Generation、Count、Size 和 Index Range。

11. 三类 History Profile

Profile	保存内容	保证
lightweight	报告、摘要、诊断、元数据、标注、Highlight、Execution Contract；不保存原始 CT	Documentary Reproducibility
full_archive	在 lightweight 基础上保存原始 CT 和选定 Artifact	Documentary Reproducibility
re_executable	固定 CT、Checkpoint、Dataset、Container、Tokenizer、Qdrant 或重建材料、Execution Contract	Executable Reproducibility

```
history:
  default_profile: lightweight
  allow_full_archive: true
  allow_re_executable: false
```

re_executable 默认关闭，必须由管理员显式启用。

12. Storage Reservation

IMP-STOR-001：大文件操作必须先预留空间

- History Snapshot
- History Chunk Storage
- Dataset Import
- Qdrant Build
- Backup
- Restore
- Export

12.1 storage_reservations

```
CREATE TABLE storage_reservations (
  id UUID PRIMARY KEY,
  reservation_id VARCHAR(128) UNIQUE NOT NULL,
  reservation_type VARCHAR(64) NOT NULL,
  workspace_id VARCHAR(128),
  operation_id VARCHAR(128),
  storage_backend VARCHAR(64) NOT NULL,
  filesystem_identity VARCHAR(256) NOT NULL,
  reserved_bytes BIGINT NOT NULL,
  consumed_bytes BIGINT NOT NULL DEFAULT 0,
  reservation_status VARCHAR(32) NOT NULL,
  owner_instance_id VARCHAR(128),
  lease_expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  released_at TIMESTAMPTZ
);
```

IMP-STOR-002：原子配额检查

必须锁定配额行，计算 Active Reservation，再插入新 Reservation。禁止“先查可用空间、后无锁插入”。

IMP-STOR-003：按文件系统分别预留

- workspace_storage
- postgres_storage
- qdrant_storage
- backup_storage
- wsl_virtual_disk

13. Backup Maintenance 完整状态机

IMP-BACK-001：状态定义

requested -> freezing -> frozen -> backup_running -> verification_running -> completed -> releasing -> released
异常路径：任何关键阶段 -> recovery_required / failed

状态	说明
requested	管理员已请求
freezing	等待 Quiescence Barrier
frozen	写入已冻结
backup_running	执行 Dump / Snapshot
verification_running	校验 Hash 和 Manifest
recovery_required	异常后待恢复判断
completed	备份成功
failed	备份失败
releasing	解除维护
released	恢复写入

IMP-BACK-002：Lease 过期不自动开放

Maintenance Lease 过期时进入 recovery_required。Reconciler 必须检查 PostgreSQL Dump、Qdrant Snapshot、Backup Epoch、Storage Reservation、Manifest 和 Hash，再决定是否可以释放。

IMP-BACK-003 : Maintenance Owner Fencing

```
maintenance_owner + maintenance_lease_revision
```

14. Qdrant Collection Content Digest**IMP-DATA-001 : 确定性内容清单**

```
UTF8(point_id) + NUL + lowercase_vector_sha256 + NUL + lowercase_payload_sha256 + LF
```

- Point ID 按规范排序
- Vector 使用 IEEE 754 float32 little-endian
- 固定维度
- 禁止 NaN / Inf
- Payload 使用 Canonical JSON

IMP-DATA-002 : Restore 验证

- Collection Metadata
- Point Count
- Vector Dimension
- Distance Metric
- Config Hash
- Content Manifest Hash
- 确定性抽样

15. Clause-level Conformance**IMP-GOV-001 : 逐条证据**

```
{
  "clause_id": "IMP-EXEC-010",
  "status": "implemented",
  "evidence": {
    "code_paths": [],
    "test_ids": [],
    "migration_ids": [],
    "config_paths": []
  },
  "deviation": null
}
```

状态	说明
implemented	已完整实现
partially_implemented	仅部分实现
not_implemented	未实现
not_applicable	经明确论证不适用
deviation_approved	偏差经批准

IMP-GOV-002 : Mandatory Clause 规则

Freeze Run 前，所有 Mandatory Clause 必须是 implemented，或存在经过批准且不削弱架构安全保证的 deviation_approved。安全关键条款不得通过偏差绕过。

16. 实现完成判定

- Dead Letter 能持久化、审计并阻止自动重放
- 重复消息只在 Owner、Revision、Generation 全部匹配时安全 ACK
- Recovery 预算区分网络、Lease、GPU 与业务 Attempt
- GPU Worker 只有在 execution_started 持久事务提交后才能 begin_execution
- Snapshot 和 Workspace 使用独立引用生命周期
- Chunk Verification 使用 Seal Token 而非长事务
- History Profile 与可复现能力严格对应
- 所有大文件操作先完成持久空间预留
- Backup Maintenance 有完整恢复状态机和 Fencing
- Qdrant Restore 可验证确定性内容 Digest
- Implementation Conformance Report 逐 Clause 提供证据

```
Architecture Bundle = v2.4.5  
Implementation Appendix = v1.1  
Status = implementation_ready  
Architecture Upgrade Required = no
```